TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline – Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the Functional Requirements Document into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade ETL solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregates.

Scope

This TRD provides technical specifications for:

- AWS Glue job configuration and runtime parameters
- S3 source and target path specifications
- Data schema definitions for all source and target entities
- Transformation logic implementation details
- Apache Hudi table configuration for SCD Type 2 implementation
- AWS Glue Data Catalog table registration specifications
- Data quality validation rules and error handling procedures
- Incremental processing logic and change detection mechanisms
- Aggregation computation specifications
- Logging and monitoring requirements

Technical components covered:

- AWS Glue ETL jobs (Python/PySpark)
- AWS Glue Data Catalog
- Amazon S3 (source and target storage)
- Apache Hudi (for SCD Type 2 implementation)
- Amazon Athena (query interface)
- AWS CloudWatch (logging and monitoring)

Out of scope:

- Infrastructure provisioning (CloudFormation/Terraform templates)

- IAM role and policy definitions
- Network configuration and VPC setup
- Cost optimization strategies
- Performance tuning beyond standard best practices
- Disaster recovery and backup procedures
- CI/CD pipeline configuration

Assumptions

- Data Format Assumptions:
- CSV files use comma delimiter with double-quote text qualifier
- CSV files contain header row as first line
- Date fields in order data are in ISO 8601 format (YYYY-MM-DD) or parseable standard format
- Numeric fields (PricePerUnit, Qty) are in standard decimal notation
- String "Null" (case-sensitive) represents null values in source data
- Empty strings and actual NULL values also represent missing data

- Data Type Assumptions (inferred from context, not specified in FRD):
- CustId: String (alphanumeric identifier)
- OrderId: String (alphanumeric identifier)
- Name: String
- EmailId: String
- Region: String
- ItemName: String
- PricePerUnit: Decimal(10,2)
- Qty: Integer
- Date: Date
- TotalAmount: Decimal(18,2)
- IsActive: Boolean
- StartDate: Timestamp
- EndDate: Timestamp (nullable)
- OpTs: Timestamp

- Processing Assumptions:
- CustId is unique identifier for customer records (primary key)
- OrderId is unique identifier for order records (primary key)
- Customer changes are detected by comparing Name, EmailId, or Region fields
- OpTs (operation timestamp) is generated using current system timestamp at processing time

• StartDate and EndDate for SCD2 use UTC timezone

• IsActive flag: True (1) for current records, False (0) for historical records

• Business key for SCD2 matching: CustId + OrderId combination

• Duplicate detection uses all columns for comparison

• Join operations use inner join unless otherwise specified

• Infrastructure Assumptions:

• AWS Glue job runs in Glue 4.0 or higher runtime environment

• Hudi library version 0.12.0 or higher is available

• Glue Data Catalog database "gen_ai_poc_databrickscoe" exists

• S3 bucket "adif-sdlc" exists with appropriate folder structure

• Glue job has IAM permissions for S3 read/write, Glue Catalog operations, and CloudWatch logging

• Sufficient Glue DPU allocation for data volume (minimum 2 DPUs recommended)

• Operational Assumptions:

• Source files arrive in complete state (no partial file processing)

• File naming convention allows for chronological ordering if needed

• No concurrent writes to same target tables from multiple jobs

• Athena query users have appropriate permissions to query catalog tables

•

2. Technical Requirements

TR-INGEST-001: Customer Data Ingestion from S3

• Technical Requirement ID: TR-INGEST-001

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Implement S3 CSV file reader to load customer master data into AWS Glue DynamicFrame for downstream processing.

• Source Details:

• Source Type: Amazon S3

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV

• File Pattern: .csv (all CSV files in directory)

• Delimiter: Comma (,)

• Header: Present (first row)

• Text Qualifier: Double quote (")

• Encoding: UTF-8

• Target Details:

• Target Type: In-memory DynamicFrame

• Target Name: customer_raw_df

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession

2. Use `glueContext.create_dynamic_frame.from_options()` method

3. Specify connection type: "s3"

4. Specify format: "csv"

5. Set format options:

• withHeader: "true"

• separator: ","

• quoteChar: "\""

• escapeChar: "\\"

6. Set connection options:

• paths: ["s3://adif-sdlc/sdlc_wizard/customerdata/"]

• recurse: True (to read all files in directory)

7. Load into DynamicFrame named customer_raw_df

8. Log record count using `customer_raw_df.count()`

• Validation Rules:

• Verify S3 path exists before read operation

• Validate that at least one CSV file is present in source path

• Confirm header row contains expected column names (CustId, Name, EmailId, Region)

• Verify record count > 0 after load

• Error Handling and Logging:

• S3 Path Not Found: Log error "Source path s3://adif-sdlc/sdlc_wizard/customerdata/ does not exist" and raise exception to fail job

• No Files Found: Log warning "No CSV files found in s3://adif-sdlc/sdlc_wizard/customerdata/" and exit job with status code 0

• CSV Parse Error: Log error with file name and line number, raise exception to fail job

• Schema Mismatch: Log error "CSV header does not match expected schema" and fail job

• Success Log: Log "Successfully loaded {count} customer records from S3"

• Runtime Parameters:

• `--SOURCE_CUSTOMER_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/ (default, overridable)

• `--CSV_SEPARATOR`: "," (default)

• `--CSV_HEADER`: "true" (default)

• Operational Notes:

• Read operation should use Glue's optimized CSV reader for performance

• Consider enabling pushdown predicates if filtering is needed in future

• Monitor CloudWatch metrics for read throughput and duration

•

TR-INGEST-002: Order Data Ingestion from S3

• Technical Requirement ID: TR-INGEST-002

• Related Functional Requirement ID(s): FR-INGEST-002

• Objective:

Implement S3 CSV file reader to load order transaction data into AWS Glue DynamicFrame for downstream processing.

• Source Details:

• Source Type: Amazon S3

• Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/

• Source Format: CSV

• File Pattern: .csv (all CSV files in directory)

• Delimiter: Comma (,)

• Header: Present (first row)

• Text Qualifier: Double quote (")

• Encoding: UTF-8

• Target Details:

• Target Type: In-memory DynamicFrame

• Target Name: order_raw_df

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession

2. Use `glueContext.create_dynamic_frame.from_options()` method

3. Specify connection type: "s3"

4. Specify format: "csv"

5. Set format options:

• withHeader: "true"

• separator: ","

• quoteChar: "\""

• escapeChar: "\\"

• inferSchema: "true" (to properly type numeric and date fields)

6. Set connection options:

• paths: ["s3://adif-sdlc/sdlc_wizard/orderdata/"]

• recurse: True

7. Load into DynamicFrame named order_raw_df

8. Log record count using `order_raw_df.count()`

• Validation Rules:

• Verify S3 path exists before read operation

• Validate that at least one CSV file is present in source path

• Confirm header row contains expected column names (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)

• Verify record count > 0 after load

• Validate PricePerUnit and Qty are numeric types

• Validate Date field is parseable as date type

• Error Handling and Logging:

• S3 Path Not Found: Log error "Source path s3://adif-sdlc/sdlc_wizard/orderdata/ does not exist" and raise exception to fail job

• No Files Found: Log warning "No CSV files found in s3://adif-sdlc/sdlc_wizard/orderdata/" and exit job with status code 0

• CSV Parse Error: Log error with file name and line number, raise exception to fail job

• Schema Mismatch: Log error "CSV header does not match expected schema" and fail job

• Type Conversion Error: Log error "Failed to convert PricePerUnit/Qty/Date to expected type" and fail job

• Success Log: Log "Successfully loaded {count} order records from S3"

• Runtime Parameters:

• `--SOURCE_ORDER_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/ (default, overridable)

• `--CSV_SEPARATOR`: "," (default)

• `--CSV_HEADER`: "true" (default)

• Operational Notes:

• Read operation should use Glue's optimized CSV reader for performance

• Date parsing should handle common formats automatically via inferSchema

• Monitor CloudWatch metrics for read throughput and duration

•

TR-CLEAN-001: Customer Data Quality Processing

• Technical Requirement ID: TR-CLEAN-001

• Related Functional Requirement ID(s): FR-CLEAN-001

• Objective:

Implement data quality transformations to remove null values (string "Null" and actual NULL) and duplicate records from customer dataset.

• Source Details:

• Source Type: In-memory DynamicFrame

• Source Name: customer_raw_df (from TR-INGEST-001)

• Target Details:

• Target Type: In-memory DynamicFrame

• Target Name: customer_clean_df

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	Yes (before cleaning)	
	Name	String	Variable	Yes (before cleaning)	
	EmailId	String	Variable	Yes (before cleaning)	
	Region	String	Variable	Yes (before cleaning)	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame: `customer_df = customer_raw_df.toDF()`

2. Remove string "Null" values:

• Apply filter condition for each column: `col != "Null"`

• Combined filter: `customer_df.filter((col("CustId") != "Null") & (col("Name") != "Null") & (col("EmailId") != "Null") & (col("Region") != "Null"))`

3. Remove actual NULL values:

• Apply `dropna()` method with parameter `how="any"` to remove rows with any null values

• `customer_df = customer_df.dropna(how="any")`

4. Remove duplicates:

• Apply `dropDuplicates()` method without parameters to consider all columns

• `customer_df = customer_df.dropDuplicates()`

5. Convert back to DynamicFrame: `customer_clean_df = DynamicFrame.fromDF(customer_df, glueContext, "customer_clean_df")`

6. Log metrics:

• Original record count

• Records removed due to "Null" string

• Records removed due to NULL values

• Duplicate records removed

• Final clean record count

• Validation Rules:

• Verify final record count > 0 (at least some valid records remain)

• Confirm no null values exist in final dataset

• Confirm no duplicate records exist in final dataset

• Log warning if more than 50% of records are removed

• Error Handling and Logging:

• All Records Removed: Log warning "All customer records removed during cleaning. Original count: {original_count}" and halt downstream processing

• High Removal Rate: If removal rate > 50%, log warning "High data quality issue: {percent}% of customer records removed"

• Success Log: Log "Customer data cleaning complete. Original: {original_count}, Removed (Null string): {null_string_count}, Removed (NULL): {null_count}, Removed (Duplicates): {dup_count}, Final: {final_count}"

• Runtime Parameters:

• `--NULL_STRING_VALUE`: "Null" (default, case-sensitive)

• `--DUPLICATE_CHECK_COLUMNS`: "all" (default, can specify subset)

• Operational Notes:

• Null removal should be performed before duplicate removal for efficiency

• Consider caching DataFrame if multiple operations are performed

• Monitor memory usage for large datasets during duplicate removal

•

TR-CLEAN-002: Order Data Quality Processing

• Technical Requirement ID: TR-CLEAN-002

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Implement data quality transformations to remove null values (string "Null" and actual NULL) and duplicate records from order dataset.

• Source Details:

• Source Type: In-memory DynamicFrame

• Source Name: order_raw_df (from TR-INGEST-002)

• Target Details:

• Target Type: In-memory DynamicFrame

• Target Name: order_clean_df

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	Yes (before cleaning)	
	ItemName	String	Variable	Yes (before cleaning)	
	PricePerUnit	Decimal	10,2	Yes (before cleaning)	
	Qty	Integer	-	Yes (before cleaning)	
	Date	Date	-	Yes (before cleaning)	
	CustId	String	Variable	Yes (before cleaning)	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame: `order_df = order_raw_df.toDF()`

2. Remove string "Null" values:

• For string columns (OrderId, ItemName, CustId): apply filter `col != "Null"`

• For numeric/date columns: string "Null" should not appear if schema inference worked correctly, but apply cast validation

• Combined filter: `order_df.filter((col("OrderId") != "Null") & (col("ItemName") != "Null") & (col("CustId") != "Null"))`

3. Remove actual NULL values:

• Apply `dropna()` method with parameter `how="any"`

• `order_df = order_df.dropna(how="any")`

4. Remove duplicates:

• Apply `dropDuplicates()` method without parameters to consider all columns

• `order_df = order_df.dropDuplicates()`

5. Convert back to DynamicFrame: `order_clean_df = DynamicFrame.fromDF(order_df, glueContext, "order_clean_df")`

6. Log metrics:

• Original record count

• Records removed due to "Null" string

• Records removed due to NULL values

• Duplicate records removed

• Final clean record count

• Validation Rules:

• Verify final record count > 0

• Confirm no null values exist in final dataset

• Confirm no duplicate records exist in final dataset

• Validate PricePerUnit > 0 (log warning for non-positive values but do not remove)

• Validate Qty > 0 (log warning for non-positive values but do not remove)

• Log warning if more than 50% of records are removed

• Error Handling and Logging:

• All Records Removed: Log warning "All order records removed during cleaning. Original count: {original_count}" and halt downstream processing

• High Removal Rate: If removal rate > 50%, log warning "High data quality issue: {percent}% of order records removed"

• Invalid Values: Log warning "Found {count} orders with PricePerUnit <= 0" and "Found {count} orders with Qty <= 0"

• Success Log: Log "Order data cleaning complete. Original: {original_count}, Removed (Null string): {null_string_count}, Removed (NULL): {null_count}, Removed (Duplicates): {dup_count}, Final: {final_count}"

• Runtime Parameters:

• `--NULL_STRING_VALUE`: "Null" (default, case-sensitive)

• `--DUPLICATE_CHECK_COLUMNS`: "all" (default)

• `--VALIDATE_POSITIVE_VALUES`: "true" (default, enables validation warnings)

• Operational Notes:

• Null removal should be performed before duplicate removal for efficiency

• Consider caching DataFrame if multiple operations are performed

• Monitor memory usage for large datasets during duplicate removal

• Date validation is implicit through schema enforcement

•

TR-CATALOG-001: Customer Table Registration in Glue Data Catalog

• Technical Requirement ID: TR-CATALOG-001

• Related Functional Requirement ID(s): FR-CATALOG-001

• Objective:

Register cleaned customer dataset as a queryable table in AWS Glue Data Catalog to enable Athena-based SQL queries and metadata management.

• Source Details:

• Source Type: In-memory DynamicFrame

• Source Name: customer_clean_df (from TR-CLEAN-001)

• Target Details:

• Target Type: AWS Glue Data Catalog Table

• Target Database: gen_ai_poc_databrickscoe

• Target Table: sdlc_wizard_customer

• Storage Format: Parquet (recommended for Athena performance)

• Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_customer/ (inferred, not specified in FRD)

• Compression: Snappy (default for Parquet)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Output Schema:

Same as Input Schema (direct registration)

• Processing / Transformation Logic:

1. Use `glueContext.write_dynamic_frame.from_catalog()` method or `glueContext.getSink()` approach

2. Specify database: "gen_ai_poc_databrickscoe"

3. Specify table name: "sdlc_wizard_customer"

4. Set format: "parquet"

5. Set update behavior: "overwrite" (for initial load) or "append" (for incremental)

6. Enable schema inference and automatic catalog update

7. Write customer_clean_df to catalog

8. Verify table registration in Glue Data Catalog

9. Log table location and record count

• Validation Rules:

• Verify database "gen_ai_poc_databrickscoe" exists before write operation

• Confirm table is created/updated successfully in Glue Data Catalog

• Validate table schema matches input DynamicFrame schema

• Verify table is queryable via Athena after registration

• Error Handling and Logging:

• Database Not Found: Log error "Glue Data Catalog database 'gen_ai_poc_databrickscoe' does not exist" and fail job

• Write Permission Error: Log error "Insufficient permissions to write to Glue Data Catalog or S3 location" and fail job

• Schema Conflict: Log error "Schema conflict detected for table sdlc_wizard_customer" and fail job

• Success Log: Log "Successfully registered {count} customer records to catalog table gen_ai_poc_databrickscoe.sdlc_wizard_customer at location {s3_path}"

• Runtime Parameters:

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_CUSTOMER`: sdlc_wizard_customer (default)

• `--CATALOG_FORMAT`: parquet (default)

• `--CATALOG_UPDATE_BEHAVIOR`: overwrite (default for full refresh)

• Operational Notes:

• Parquet format is recommended for Athena query performance and cost efficiency

• Consider partitioning strategy if customer data volume grows significantly (not specified in FRD)

• Table should be registered with appropriate metadata for data governance

• Enable Glue Data Catalog versioning for schema evolution tracking

•

TR-CATALOG-002: Order Table Registration in Glue Data Catalog

• Technical Requirement ID: TR-CATALOG-002

• Related Functional Requirement ID(s): FR-CATALOG-002

• Objective:

Register cleaned order dataset as a queryable table in AWS Glue Data Catalog to enable Athena-based SQL queries and metadata management.

• Source Details:

• Source Type: In-memory DynamicFrame

• Source Name: order_clean_df (from TR-CLEAN-002)

• Target Details:

• Target Type: AWS Glue Data Catalog Table

• Target Database: gen_ai_poc_databrickscoe

• Target Table: sdlc_wizard_order

• Storage Format: Parquet (recommended for Athena performance)

• Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_order/ (inferred, not specified in FRD)

• Compression: Snappy (default for Parquet)

• Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema:

Same as Input Schema (direct registration)

• Processing / Transformation Logic:

1. Use `glueContext.write_dynamic_frame.from_catalog()` method or `glueContext.getSink()` approach

2. Specify database: "gen_ai_poc_databrickscoe"

3. Specify table name: "sdlc_wizard_order"

4. Set format: "parquet"

5. Set update behavior: "overwrite" (for initial load) or "append" (for incremental)

6. Enable schema inference and automatic catalog update

7. Write order_clean_df to catalog

8. Verify table registration in Glue Data Catalog

9. Log table location and record count

• Validation Rules:

• Verify database "gen_ai_poc_databrickscoe" exists before write operation

• Confirm table is created/updated successfully in Glue Data Catalog

• Validate table schema matches input DynamicFrame schema

• Verify table is queryable via Athena after registration

• Validate Date column is properly typed as date in catalog

• Error Handling and Logging:

• Database Not Found: Log error "Glue Data Catalog database 'gen_ai_poc_databrickscoe' does not exist" and fail job

• Write Permission Error: Log error "Insufficient permissions to write to Glue Data Catalog or S3 location" and fail job

• Schema Conflict: Log error "Schema conflict detected for table sdlc_wizard_order" and fail job

• Success Log: Log "Successfully registered {count} order records to catalog table gen_ai_poc_databrickscoe.sdlc_wizard_order at location {s3_path}"

• Runtime Parameters:

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_ORDER`: sdlc_wizard_order (default)

• `--CATALOG_FORMAT`: parquet (default)

• `--CATALOG_UPDATE_BEHAVIOR`: overwrite (default for full refresh)

• Operational Notes:

• Parquet format is recommended for Athena query performance and cost efficiency

• Consider partitioning by Date field for query performance optimization (not specified in FRD)

• Table should be registered with appropriate metadata for data governance

• Enable Glue Data Catalog versioning for schema evolution tracking

•

TR-SCD2-001: SCD Type 2 Implementation for Order Summary Using Hudi

• Technical Requirement ID: TR-SCD2-001

• Related Functional Requirement ID(s): FR-SCD2-001

• Objective:

Implement Slowly Changing Dimension Type 2 logic using Apache Hudi to maintain complete historical lineage of customer-order relationships with active/inactive flags and effective date ranges.

• Source Details:

• Source Type: In-memory DynamicFrame (joined dataset)

• Source Name: customer_order_joined_df

• Source Components:

• customer_clean_df (from TR-CLEAN-001)

• order_clean_df (from TR-CLEAN-002)

• Target Details:

• Target Type: Apache Hudi Table

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Hudi Table Type: Copy-On-Write (COW) (recommended for read-heavy workloads)

• Hudi Record Key: CustId + OrderId (composite key)

• Hudi Precombine Key: OpTs (to determine latest record)

• Hudi Partition Path: None (not specified in FRD, can be added for performance)

• Input Schema (Joined Dataset):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source	
	---	---	---	---	---	
	CustId	String	Variable	No	Customer & Order	
	OrderId	String	Variable	No	Order	
	Name	String	Variable	No	Customer	
	EmailId	String	Variable	No	Customer	
	Region	String	Variable	No	Customer	
	ItemName	String	Variable	No	Order	
	PricePerUnit	Decimal	10,2	No	Order	
	Qty	Integer	-	No	Order	
	Date	Date	-	No	Order	

• Output Schema (Hudi Table):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Derivation	
	---	---	---	---	---	
	CustId	String	Variable	No	Direct map from Customer	
	OrderId	String	Variable	No	Direct map from Order	

	Name	String	Variable	No	Direct map from Customer	
	EmailId	String	Variable	No	Direct map from Customer	
	Region	String	Variable	No	Customer	
	ItemName	String	Variable	No	Direct map from Order	
	PricePerUnit	Decimal	10,2	No	Direct map from Order	
	Qty	Integer	-	No	Direct map from Order	
	Date	Date	-	No	Direct map from Order	
	IsActive	Boolean	-	No	System-generated (SCD2 flag)	
	StartDate	Timestamp	-	No	System-generated (SCD2 effective start)	
	EndDate	Timestamp	-	Yes	System-generated (SCD2 effective end, null for active)	
	OpTs	Timestamp	-	No	System-generated (operation timestamp)	

• Processing / Transformation Logic:

• Step 1: Join Customer and Order Data

1. Convert customer_clean_df and order_clean_df to Spark DataFrames

2. Perform inner join on CustId: `customer_order_df = customer_df.join(order_df, "CustId", " inner")`

3. Log join statistics (customer count, order count, joined record count)

4. Identify orphaned orders (orders with CustId not in customer table) and log warning

• Step 2: Read Existing Hudi Table (if exists)

1. Check if Hudi table exists at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

2. If exists, read existing Hudi table into DataFrame: `existing_hudi_df = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")`

3. Filter for active records only: `active_records_df = existing_hudi_df.filter(col("IsActive") == True)`

4. If table does not exist, proceed with initial load logic

• Step 3: Detect Changes and Apply SCD2 Logic

1. Define business key: composite of CustId + OrderId

2. Define change detection columns: Name, EmailId, Region

3. For each record in customer_order_df:

• Generate composite key: `concat(CustId, "_", OrderId)`

• Check if matching active record exists in active_records_df

• If no active record exists (new record):

• Add SCD2 columns:

• IsActive = True

• StartDate = current_timestamp()

• EndDate = null

• OpTs = current_timestamp()

• Mark for insert

• If active record exists:

• Compare change detection columns (Name, EmailId, Region)

• If change detected:

• Mark existing active record for update:

• IsActive = False

• EndDate = current_timestamp()

• Create new record with updated values:

• IsActive = True

• StartDate = current_timestamp()

• EndDate = null

• OpTs = current_timestamp()

• If no change detected:

• Skip processing (no action needed)

• Step 4: Prepare Upsert DataFrame

1. Combine records marked for insert and update into single DataFrame

2. Add Hudi metadata columns:

• _hoodie_record_key: concat(CustId, "_", OrderId)

• _hoodie_partition_path: "" (empty if not partitioned)

3. Ensure OpTs is set for all records (used as precombine key)

• Step 5: Write to Hudi Table

1. Configure Hudi write options:

```python
hudi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'CustId,OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.upsert.shuffle.parallelism': 2,
'hoodie.insert.shuffle.parallelism': 2
}
```

```

2. Write DataFrame to Hudi:

```python
upsert_df.write.format("hudi") \
.options(hudi_options) \
.mode("append") \
.save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

• Step 6: Register/Update Glue Catalog

1. Use Hudi's Glue Catalog sync utility or manual registration

2. Register table as: gen_ai_poc_databrickscoe.ordersummary

3. Enable Hudi metadata columns for time-travel queries

4. Verify table is queryable via Athena

• Validation Rules:

• Verify join produces records (no empty result set)

• Validate that all active records have EndDate = null

• Validate that all inactive records have EndDate populated

• Confirm no duplicate active records for same business key

• Validate OpTs is populated for all records

• Verify StartDate <= EndDate for all inactive records

• Error Handling and Logging:

• Empty Join Result: Log warning "Customer-Order join produced 0 records" and skip Hudi write

• Orphaned Orders: Log warning "Found {count} orders with CustId not in customer table: {custid_list}"

• Hudi Write Failure: Log error "Hudi upsert operation failed: {error_message}" and fail job

• Catalog Registration Failure: Log error "Failed to register Hudi table in Glue Catalog" and fail job

• Success Log: Log "SCD2 processing complete. New records: {new_count}, Updated records: {update_count}, Total active records: {active_count}, Total historical records: {historical_count}"

• Runtime Parameters:

• `--HUDI_TABLE_NAME`: ordersummary (default)

• `--HUDI_TABLE_TYPE`: COPY_ON_WRITE (default)

• `--HUDI_RECORD_KEY`: CustId,OrderId (default)

• `--HUDI_PRECOMBINE_KEY`: OpTs (default)

• `--HUDI_OPERATION`: upsert (default)
```

- `--TARGET_HUDI_PATH`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (default)
- `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)
- `--CATALOG_TABLE_ORDERSUMMARY`: ordersummary (default)

- Operational Notes:
- Hudi COW table type is recommended for read-heavy analytics workloads
- Consider partitioning by Date or Region for large datasets (not specified in FRD)
- Hudi's upsert operation handles both inserts and updates efficiently
- Time-travel queries can be performed using Hudi's incremental query capabilities
- Monitor Hudi compaction and cleaning operations for optimal performance
- Ensure sufficient Glue DPU allocation for Hudi write operations (minimum 2 DPUs)
-

TR-INCR-001: Incremental SCD2 Processing Based on Customer Changes

- Technical Requirement ID: TR-INCR-001

- Related Functional Requirement ID(s): FR-INCR-001

- Objective:

Implement change detection mechanism to identify modified customer records and trigger incremental SCD2 updates for affected customer-order combinations, optimizing processing performance.

- Source Details:
- Source Type: In-memory DynamicFrame (new customer data)
- Source Name: customer_clean_df (from TR-CLEAN-001)
- Reference Source: Existing ordersummary Hudi table (from TR-SCD2-001)

- Target Details:
- Target Type: Apache Hudi Table (incremental update)
- Target Database: gen_ai_poc_databrickscoe
- Target Table: ordersummary
- Target S3 Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Input Schema:

Same as TR-SCD2-001 (customer and order data)

- Output Schema:

Same as TR-SCD2-001 (ordersummary Hudi table)

- Processing / Transformation Logic:

• Step 1: Read Previous Customer State

1. Read existing ordersummary Hudi table

2. Extract unique customer records (CustId, Name, EmailId, Region) from active records

3. Create previous_customer_df with distinct customer attributes

• Step 2: Detect Customer Changes

1. Perform full outer join between customer_clean_df and previous_customer_df on CustId

2. Identify change types:

• New Customer: CustId exists in customer_clean_df but not in previous_customer_df

• Updated Customer: CustId exists in both, but Name, EmailId, or Region differs

• Unchanged Customer: CustId exists in both with identical attributes

3. Create changed_customers_df containing only new and updated customers

4. Log change statistics:

• New customers count

• Updated customers count

• Unchanged customers count

• Step 3: Filter Affected Orders

1. Extract CustId list from changed_customers_df

2. Filter order_clean_df to include only orders for changed customers:

`affected_orders_df = order_clean_df.filter(col("CustId").isin(changed_custid_list))`

3. Log affected orders count

• Step 4: Apply SCD2 Logic for Affected Records Only

1. Join changed_customers_df with affected_orders_df on CustId

2. Apply same SCD2 logic as TR-SCD2-001, but only for affected customer-order combinations

3. For unchanged customers, skip processing entirely (no read or write operations)

• Step 5: Incremental Hudi Upsert

1. Use same Hudi write configuration as TR-SCD2-001

2. Set operation to "upsert" to handle both new and updated records

3. Hudi will automatically handle:

• Inserting new records

• Updating existing records based on record key

• Maintaining historical versions

• Validation Rules:

• Verify change detection logic correctly identifies new, updated, and unchanged customers

• Validate that only affected orders are processed

• Confirm no unintended updates to unchanged customer-order records

• Verify incremental processing produces same result as full processing for affected records

• Error Handling and Logging:

• No Changes Detected: Log info "No customer changes detected. Skipping incremental processing." and exit gracefully

• Change Detection Failure: Log error "Failed to detect customer changes: {error_message}" and fall back to full processing per TR-SCD2-001

• Affected Orders Not Found: Log warning "No orders found for changed customers" and skip processing

• Success Log: Log "Incremental SCD2 processing complete. Changed customers: {changed_count}, Affected orders: {order_count}, Records updated: {update_count}"

• Runtime Parameters:

• `--INCREMENTAL_MODE`: true (default, set to false for full refresh)

• `--CHANGE_DETECTION_COLUMNS`: Name,EmailId,Region (default)

• All Hudi parameters from TR-SCD2-001

• Operational Notes:

• Incremental processing significantly reduces processing time for large datasets

• Change detection should be performed at customer level, not order level

• If change detection fails or produces unexpected results, fall back to full processing

• Monitor incremental processing performance and adjust DPU allocation as needed

• Consider implementing watermark-based incremental processing for very large datasets (not specified in FRD)

•

TR-AGG-001: Customer Aggregate Spend Calculation

• Technical Requirement ID: TR-AGG-001

• Related Functional Requirement ID(s): FR-AGG-001

• Objective:

Generate customer-level spending aggregates by joining customer and order data, calculating total spend per customer per date, and producing an analytics-ready table for business intelligence consumption.

• Source Details:

• Source Type: In-memory DynamicFrame (joined dataset)

• Source Components:

• customer_clean_df (from TR-CLEAN-001)

• order_clean_df (from TR-CLEAN-002)

• Target Details:

• Target Type: AWS Glue Data Catalog Table

• Target Database: gen_ai_poc_databrickscoe

• Target Table: customeraggregatespend

• Target S3 Location: s3://adif-sdlc/analytics/customeraggregatespend/

• Storage Format: Parquet (recommended for analytics)

• Compression: Snappy

• Input Schema (Joined Dataset):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source	
	---	---	---	---	---	
	CustId	String	Variable	No	Customer & Order	
	Name	String	Variable	No	Customer	
	PricePerUnit	Decimal	10,2	No	Order	
	Qty	Integer	-	No	Order	
	Date	Date	-	No	Order	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Derivation	
	---	---	---	---	---	
	Name	String	Variable	No	Direct map from Customer	
	TotalAmount	Decimal	18,2	No	Calculated: SUM(PricePerUnit * Qty)	
	Date	Date	-	No	Direct map from Order	

• Processing / Transformation Logic:

• Step 1: Join Customer and Order Data

1. Convert customer_clean_df and order_clean_df to Spark DataFrames

2. Perform inner join on CustId: `customer_order_df = customer_df.join(order_df, "CustId", " inner")`

3. Select required columns: CustId, Name, PricePerUnit, Qty, Date

• Step 2: Calculate Line Item Amount

1. Add calculated column: `LineAmount = PricePerUnit Qty`

2. Validate LineAmount is non-negative (log warning for negative values)

• Step 3: Aggregate by Customer and Date

1. Group by Name and Date

2. Calculate aggregate: `TotalAmount = SUM(LineAmount)`

3. Result schema: Name, Date, TotalAmount

• Step 4: Sort Results

1. Sort by Name (ascending), Date (ascending) for consistent output

2. Optional: Add row_number or rank for additional analytics

• Step 5: Write to S3 and Register in Catalog

1. Write aggregated DataFrame to S3 location: s3://adif-sdlc/analytics/customeraggregatespend/

2. Use Parquet format with Snappy compression

3. Set write mode: "overwrite" (for full refresh) or "append" (for incremental)

4. Register table in Glue Data Catalog:

• Database: gen_ai_poc_databrickscoe

• Table: customeraggregatespend

5. Enable table for Athena queries

• Validation Rules:

• Verify join produces records (no empty result set)

• Validate TotalAmount >= 0 for all records (log warning for negative values)

• Confirm aggregation produces expected record count (one row per customer per date)

• Verify no null values in output columns

• Error Handling and Logging:

• Empty Join Result: Log warning "Customer-Order join for aggregation produced 0 records" and create empty table

• Negative TotalAmount: Log warning "Found {count} records with negative TotalAmount: {details}"

• Aggregation Failure: Log error "Aggregation calculation failed: {error_message}" and fail job

• Write Failure: Log error "Failed to write aggregate data to S3: {error_message}" and fail job

• Catalog Registration Failure: Log error "Failed to register customeraggregatespend table in Glue Catalog" and fail job

• Success Log: Log "Customer aggregate spend calculation complete. Total customers: {customer_count}, Total records: {record_count}, Date range: {min_date} to {max_date}"

• Runtime Parameters:

• `--TARGET_AGGREGATE_PATH`: s3://adif-sdlc/analytics/customeraggregatespend/ (default)

• `--CATALOG_DATABASE`: gen_ai_poc_databrickscoe (default)

• `--CATALOG_TABLE_AGGREGATE`: customeraggregatespend (default)

• `--AGGREGATE_FORMAT`: parquet (default)

• `--AGGREGATE_UPDATE_BEHAVIOR`: overwrite (default)

• Operational Notes:

• Aggregation should be performed after data quality checks to ensure accurate calculations

• Consider partitioning by Date for query performance optimization (not specified in FRD)

• TotalAmount precision (18,2) accommodates large spending values

• Parquet format with Snappy compression provides optimal balance of compression and query performance

• Monitor aggregation performance for large datasets and adjust DPU allocation as needed

•

3. Runtime Strategy Notes

Authoring Language

• Primary Language: Python 3 (PySpark)

• Glue Version: AWS Glue 4.0 or higher

• Spark Version: 3.3.0 or higher (bundled with Glue 4.0)

• Python Libraries:

• pyspark (built-in)

• awsglue (built-in)

• boto3 (for S3 operations, built-in)

• hudi (Apache Hudi library, version 0.12.0 or higher)

Execution Strategy

• Job Type: AWS Glue ETL Job (Spark)

• Execution Mode: Standard (batch processing)

• Worker Type: G.1X or G.2X (depending on data volume)

• Number of Workers: 2-10 (auto-scaling recommended)

• Job Timeout: 2880 minutes (48 hours, adjust based on data volume)

• Max Retries: 1

• Job Bookmark: Enabled (for incremental processing support)

Glue Job Type Expectations

• Script Type: Python Shell or Spark (Spark recommended for data volume and Hudi support)

• Glue Data Catalog Integration: Required (for table registration and metadata management)

• S3 Access: Required (read from source paths, write to target paths)

- CloudWatch Logging: Enabled (for job monitoring and debugging)
- Metrics: Enabled (for performance monitoring)
- Security Configuration: Optional (for encryption at rest and in transit)
-

4. Data Design Details

Source Paths / Source Systems

- Customer Data Source: s3://adif-sdlc/sdlc_wizard/customerdata/
- Format: CSV
- Delimiter: Comma
- Header: Present
- Encoding: UTF-8
- Order Data Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Delimiter: Comma
- Header: Present
- Encoding: UTF-8

Target Paths / Target Tables

- Customer Catalog Table:
- Database: gen_ai_poc_databrickscoe
- Table: sdlc_wizard_customer
- Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_customer/ (inferred)
- Format: Parquet
- Order Catalog Table:
- Database: gen_ai_poc_databrickscoe
- Table: sdlc_wizard_order
- Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_order/ (inferred)
- Format: Parquet
- Order Summary Hudi Table:
- Database: gen_ai_poc_databrickscoe
- Table: ordersummary
- Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Format: Apache Hudi (Copy-On-Write)
- Customer Aggregate Spend Table:

- Database: gen_ai_poc_databrickscoe
- Table: customeraggregatespend
- Storage Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Format: Parquet

Catalog Registration Expectations

- All target tables must be registered in AWS Glue Data Catalog database: gen_ai_poc_databrickscoe
- Schema inference should be enabled for automatic metadata capture
- Table metadata should include:
- Column names and data types
- Storage location
- File format
- Compression codec
- Creation timestamp
- Last update timestamp
- Tables should be queryable via Amazon Athena immediately after registration
- Glue Data Catalog versioning should be enabled for schema evolution tracking

Partitioning Expectations

- Not specified in FRD, but recommended for performance:
- Order table: Consider partitioning by Date (year/month/day hierarchy)
- Order Summary Hudi table: Consider partitioning by Date or Region
- Customer Aggregate Spend table: Consider partitioning by Date
- If partitioning is implemented:
- Use Hive-style partitioning (e.g., year=2024/month=01/day=15)
- Register partition metadata in Glue Data Catalog
- Enable partition projection in Athena for query performance

File Formats / Table Formats

- Source Files: CSV (comma-delimited, with header)
- Catalog Tables: Parquet (columnar format for analytics)
- Hudi Table: Apache Hudi Copy-On-Write (for SCD Type 2 with time-travel)
- Compression: Snappy (default for Parquet and Hudi)

Update / Overwrite / Append Behavior

- Customer Catalog Table:

• Initial Load: Overwrite mode

• Incremental Load: Append mode (if incremental processing is implemented)

• Order Catalog Table:

• Initial Load: Overwrite mode

• Incremental Load: Append mode (if incremental processing is implemented)

• Order Summary Hudi Table:

• All Loads: Upsert mode (Hudi handles inserts and updates automatically)

• SCD Type 2 logic maintains historical versions

• Customer Aggregate Spend Table:

• Initial Load: Overwrite mode

• Incremental Load: Overwrite mode (full recalculation) or Append mode (if date-based incremental logic is implemented)

•

5. Processing Details

Data Quality Rules

• Null Removal:

• Remove rows containing string "Null" (case-sensitive) in any column

• Remove rows containing actual NULL values in any column

• Apply to both customer and order datasets before downstream processing

• Duplicate Removal:

• Remove duplicate rows based on all columns

• Apply after null removal for efficiency

• Log count of removed duplicates for audit purposes

• Data Type Validation:

• Validate PricePerUnit and Qty are numeric types

• Validate Date field is parseable as date type

• Log warnings for type conversion failures

• Business Rule Validation:

• Log warnings for PricePerUnit <= 0 (but do not remove)

• Log warnings for Qty <= 0 (but do not remove)

• Log warnings for negative TotalAmount in aggregates

Deduplication Rules

• Deduplication Scope: All columns

• Deduplication Method: Spark DataFrame `dropDuplicates()` method without parameters

• Deduplication Timing: After null removal, before catalog registration

• Deduplication Logging: Log count of duplicate records removed for each dataset

Join Rules

• Customer-Order Join (for SCD2 and Aggregation):

• Join Type: Inner join

• Join Key: CustId

• Join Behavior: Retain only orders with matching customer records

• Orphaned Orders: Log warning for orders with CustId not in customer table

• Customer Change Detection Join (for Incremental Processing):

• Join Type: Full outer join

• Join Key: CustId

• Join Behavior: Identify new, updated, and unchanged customers

Aggregation Rules

• Customer Aggregate Spend:

• Aggregation Level: Customer (Name) and Date

• Aggregation Function: SUM(PricePerUnit Qty)

• Output Column: TotalAmount (Decimal 18,2)

• Sorting: By Name (ascending), Date (ascending)

• Null Handling: Exclude records with null values in aggregation inputs

SCD / History Tracking Rules

• SCD Type 2 Implementation:

• Business Key: CustId + OrderId (composite key)

• Change Detection Columns: Name, EmailId, Region

• Active Record Flag: IsActive (Boolean, True for current, False for historical)

• Effective Date Range: StartDate (Timestamp, record effective start), EndDate (Timestamp, record effective end, null for active)

• Operation Timestamp: OpTs (Timestamp, system-generated at processing time)

• Precombine Key: OpTs (used by Hudi to determine latest record)

• SCD Type 2 Logic:

• New Record: Insert with IsActive=True, StartDate=current_timestamp, EndDate=null, OpTs=current_timestamp

• Changed Record:

• Update existing active record: IsActive=False, EndDate=current_timestamp

• Insert new record: IsActive=True, StartDate=current_timestamp, EndDate=null, OpTs=current_timestamp

• Unchanged Record: No action (skip processing)

• Historical Preservation:

• All versions of records are retained in Hudi table

• Time-travel queries can retrieve historical states

• Active records always have EndDate=null

• Inactive records always have EndDate populated

Null Handling

• Source Data:

• String "Null" (case-sensitive) is treated as null value and removed

• Actual NULL values are removed

• Empty strings are treated as null values and removed

• Target Data:

• No null values allowed in non-nullable columns after cleaning

• EndDate in SCD2 table is nullable (null for active records)

• Aggregation:

• Null values in aggregation inputs are excluded from calculations

• Result columns (Name, TotalAmount, Date) are non-nullable

Type Casting Expectations

• CSV to DataFrame:

• Enable schema inference for automatic type detection

• Explicitly cast columns if inference fails:

• PricePerUnit: cast to DecimalType(10,2)

• Qty: cast to IntegerType

• Date: cast to DateType using to_date() function

• SCD2 Columns:

• IsActive: cast to BooleanType

• StartDate: cast to TimestampType using current_timestamp()

• EndDate: cast to TimestampType (nullable)

• OpTs: cast to TimestampType using current_timestamp()

• Aggregation:

• TotalAmount: cast to DecimalType(18,2)

• Error Handling:

• Log errors for type casting failures

• Fail job if critical type casts fail (e.g., PricePerUnit, Qty, Date)

•

6. Traceability Matrix

D Requirement ID(s)	Description	Original Source	Technical Sourc
----------	----------	----------	----------
-INGEST-001	Customer data ingestion from S3	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdl
-INGEST-002	Order data ingestion from S3	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdl
-CLEAN-001	Customer data quality processing	customer_raw_df (from TR-INGEST-001)	customer_raw_d
-CLEAN-002	Order data quality processing	order_raw_df (from TR-INGEST-002)	order_raw_df (fr
-CATALOG-001	Customer table registration in Glue Catalog	customer_clean_df (from TR-CLEAN-001)	customer_clean_
-CATALOG-002	Order table registration in Glue Catalog	order_clean_df (from TR-CLEAN-002)	order_clean_df (
-SCD2-001	SCD Type 2 implementation for order summary	customer_clean_df + order_clean_df	customer_clean_
-INCR-001	Incremental SCD2 processing based on customer changes	customer_clean_df + existing ordersummary	customer_clean_
-AGG-001	Customer aggregate spend calculation	customer_clean_df + order_clean_df	customer_clean_

• Source Fidelity Verification:

• All technical sources match original sources from FRD

• No intermediate sources or staging tables introduced

• Source paths preserved exactly as specified in FRD

• In-memory transformations use DynamicFrames/DataFrames derived directly from original sources

•

7. ETL Mapping Requirements

TR-INGEST-001: Customer Data Ingestion

• Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	----------	----------	----------	----------	
	CustId	String	Variable	Yes	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

• Target: customer_raw_df (DynamicFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	----------	----------	----------	----------	---------	
	CustId	String	Variable	Yes	Direct map from source CustId	
	Name	String	Variable	Yes	Direct map from source Name	
	EmailId	String	Variable	Yes	Direct map from source EmailId	
	Region	String	Variable	Yes	Direct map from source Region	

•

TR-INGEST-002: Order Data Ingestion

• Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	----------	----------	----------	----------	
	OrderId	String	Variable	Yes	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

• Target: order_raw_df (DynamicFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	----------	----------	----------	----------	---------	
	OrderId	String	Variable	Yes	Direct map from source OrderId	
	ItemName	String	Variable	Yes	Direct map from source ItemName	
	PricePerUnit	Decimal	10,2	Yes	Direct map from source PricePerUnit	
	Qty	Integer	-	Yes	Direct map from source Qty	
	Date	Date	-	Yes	Direct map from source Date	
	CustId	String	Variable	Yes	Direct map from source CustId	

•

TR-CLEAN-001: Customer Data Quality Processing

• Source: customer_raw_df (from TR-INGEST-001)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	----------	----------	----------	----------	
	CustId	String	Variable	Yes	
	Name	String	Variable	Yes	

	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

• Target: customer_clean_df (DynamicFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	CustId	String	Variable	No	Direct map from source CustId (after null and duplicate removal)	
	Name	String	Variable	No	Direct map from source Name (after null and duplicate removal)	
	EmailId	String	Variable	No	Direct map from source EmailId (after null and duplicate removal)	
	Region	String	Variable	No	Direct map from source Region (after null and duplicate removal)	

•

TR-CLEAN-002: Order Data Quality Processing

• Source: order_raw_df (from TR-INGEST-002)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	Yes	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

• Target: order_clean_df (DynamicFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
	---	---	---	---	---
	OrderId	String	Variable	No	Direct map from source OrderId (after null and duplicate removal)
	ItemName	String	Variable	No	Direct map from source ItemName (after null and duplicate removal)
	PricePerUnit	Decimal	10,2	No	Direct map from source PricePerUnit (after null and duplicate removal)
	Qty	Integer	-	No	Direct map from source Qty (after null and duplicate removal)
	Date	Date	-	No	Direct map from source Date (after null and duplicate removal)
	CustId	String	Variable	No	Direct map from source CustId (after null and duplicate removal)

•

TR-CATALOG-001: Customer Table Registration

• Source: customer_clean_df (from TR-CLEAN-001)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer (Glue Catalog Table, Parquet)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	CustId	String	Variable	No	Direct map from source CustId	
	Name	String	Variable	No	Direct map from source Name	
	EmailId	String	Variable	No	Direct map from source EmailId	
	Region	String	Variable	No	Direct map from source Region	

•

TR-CATALOG-002: Order Table Registration

• Source: order_clean_df (from TR-CLEAN-002)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_order (Glue Catalog Table, Parquet)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	OrderId	String	Variable	No	Direct map from source OrderId	
	ItemName	String	Variable	No	Direct map from source ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from source PricePerUnit	
	Qty	Integer	-	No	Direct map from source Qty	
	Date	Date	-	No	Direct map from source Date	
	CustId	String	Variable	No	Direct map from source CustId	

•

TR-SCD2-001: SCD Type 2 Implementation for Order Summary

• Source: customer_clean_df (from TR-CLEAN-001) + order_clean_df (from TR-CLEAN-002)

• Joined Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Table	
	---	---	---	---	---	
	CustId	String	Variable	No	Customer & Order	
	OrderId	String	Variable	No	Order	
	Name	String	Variable	No	Customer	
	EmailId	String	Variable	No	Customer	
	Region	String	Variable	No	Customer	
	ItemName	String	Variable	No	Order	
	PricePerUnit	Decimal	10,2	No	Order	
	Qty	Integer	-	No	Order	
	Date	Date	-	No	Order	

• Target: gen_ai_poc_databrickscoe.ordersummary (Hudi Table)

lumn Name	Data Type	Length/Precision/Scale	Nullable	Mapping
---	---	---	---	---
stId	String	Variable	No	Direct map from joined source CustId
derId	String	Variable	No	Direct map from joined source OrderId
me	String	Variable	No	Direct map from joined source Name
ailId	String	Variable	No	Direct map from joined source EmailId
gion	String	Variable	No	Direct map from joined source Region
mName	String	Variable	No	Direct map from joined source ItemName
cePerUnit	Decimal	10,2	No	Direct map from joined source PricePerUnit
y	Integer	-	No	Direct map from joined source Qty
te	Date	-	No	Direct map from joined source Date
ctive	Boolean	-	No	System-generated: True for active records, False for historical
rtDate	Timestamp	-	No	System-generated: current_timestamp() at record creation
dDate	Timestamp	-	Yes	System-generated: current_timestamp() when record becomes inactive, null for act
Ts	Timestamp	-	No	System-generated: current_timestamp() at processing time

•

TR-INCR-001: Incremental SCD2 Processing

• Source: customer_clean_df (from TR-CLEAN-001) + existing ordersummary (from TR-SCD2-001)

• Source Schema: Same as TR-SCD2-001

• Target: gen_ai_poc_databrickscoe.ordersummary (Hudi Table, incremental update)

• Target Schema: Same as TR-SCD2-001

• Mapping: Same as TR-SCD2-001, but only for changed customer records

•

TR-AGG-001: Customer Aggregate Spend Calculation

• Source: customer_clean_df (from TR-CLEAN-001) + order_clean_df (from TR-CLEAN-002)

• Joined Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Table	
	---	---	---	---	---	
	CustId	String	Variable	No	Customer & Order	
	Name	String	Variable	No	Customer	
	PricePerUnit	Decimal	10,2	No	Order	
	Qty	Integer	-	No	Order	
	Date	Date	-	No	Order	

• Target: gen_ai_poc_databrickscoe.customeraggregatespend (Glue Catalog Table, Parquet)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	---	---	---	---	---	
	Name	String	Variable	No	Direct map from joined source Name	
	TotalAmount	Decimal	18,2	No	Calculated: SUM(PricePerUnit * Qty) grouped by Name and Date	
	Date	Date	-	No	Direct map from joined source Date	

•

• End of Technical Requirements Document